BrainAI

ProSuite Industrial

API & WRAPPER SUITE

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**ProHash Integration Guide**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2025-01-01	S. Köhne	Stable	Initial Release of Wrapper Documentation
2.0.0	2026-02-27	S. Köhne	Stable	Full API Integration & Streaming Guide

1 Product Overview

The **ProHash Wrapper Suite** serves as the high-level interface to the native ProHash-256 engine ("The Shredder"). It enables developers to efficiently utilize the algorithm's sponge construction without having to manage memory manually or handle bit manipulations at the C level.

Key Integration Features:

- **Deterministic Hashing:** Identical results across all platforms (x86, x64, ARM).
- **Sponge Streaming:** Processing of data volumes exceeding available RAM through sequential updates.
- **Zero-Trust Memory Management:** Automatic wiping of sensitive state data after finalization.

2 Common Fundamentals

All wrappers map the native `ProHash_Ctx` structure (136 bytes), which manages the 512-bit internal state (`belt`), the byte counter (`count`), and the input buffer.

Standard Workflow:

1. **Initialization:** Creating the context and setting the initial state (`prohash_init`).
2. **Update:** Processing data fragments sequentially (`prohash_update`).
3. **Finalization:** Applying padding and generating the 32-byte digest (`prohash_final`).

3 C# (.NET) Integration

The C# wrapper utilizes `P/Invoke` and `unsafe` blocks to manage the context directly in unmanaged memory.

- **Key Detail:** Manual memory allocation via `Marshal.AllocHGlobal` to prevent Garbage Collector (GC) relocation.
- **Example:**

```
using (var hasher = new ProHash()) {
    hasher.Update(Encoding.UTF8.GetBytes("Data"));
    byte[] result = hasher.Final();
    // result contains 32 bytes
}
```

4 Java (JNA) Integration

The Java connection is implemented via JNA with strict 8-byte alignment for the `uint64_t` counter.

- **Maven/Gradle:** Requires the `net.java.dev.jna` dependency.
- **Example:**

```
ProHash hasher = new ProHash();
hasher.update(fileContent);
byte[] digest = hasher.finalizeHash();
```

5 Python Integration

Uses `ctypes` and is platform-agnostic (loads `.dll` or `.so` files automatically).

- **Example:**

```
hasher = ProHash()
hasher.update(b"Chunk 1")
```

```
digest = hasher.finalize() # Returns 32 bytes
```

6 JavaScript (Node.js) Integration

Utilizes `ffi-napi` for native calls and `Buffer` for efficient data transfer.

- **Streaming Support:** Ideally suited for connections with `fs.createReadStream`.
- **Example:**

```
const hasher = new ProHash();
hasher.update(Buffer.from("Input"));
const hash = hasher.finalize();
```

7 VB.NET Integration

Optimized for OT (Operational Technology) systems using `MarshalAs` attributes to ensure fixed array sizes.

- **Example:**

```
Dim hasher As New ProHash()
hasher.Update(dataBytes)
Dim result = hasher.FinalizeHash()
```

8 Best Practices & Security

- **Hex Conversion:** For display purposes, use the `"x2"` format string to convert the 32-byte digest into a 64-character hex string.
- **Threading:** ProHash contexts are not thread-safe. Use a separate instance for each thread.
- **Streaming:** For files larger than 100 MB, always use the `Update` method in loops (chunks) to keep RAM consumption consistently low.

© 2026 Sascha Köhne	✉️ **Contact:** koehne83@googlemail.com	🛡️ **ProTU-Verified**
CEO / System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**